

제 1 회 Physical AI 해커톤

- LeRobot ACT 매뉴얼

제 1 회 Physical AI 해커톤 행사 자료.pdf 1.6 MiB

The 1st Physical AI Hackathon Participant Guide.pdf 139 KiB

0. 실행환경

- 우분투 22.04 - python 3.10 .0 - lerobot 0.4 .3 - Huggingface - Cuda 13.1 - Miniconda

1. 운영 아키텍처

[로컬 로봇 PC] 텔레옴 + 데이터 수집, ACT 추론 [서버 GPU (A100)] ACT 학습

2. 환경 세팅

2.0 Lerobot, 미니콘다 설치

먼저 사용자의 기존 환경을 유지하며 진행하기 위해 Miniconda 가상환경을 세팅하고 Lerobot 0.4 .3 버전을 설치한다 . # 미니콘다 설치 `wget https : // repo . anaconda . com / miniconda / Miniconda3 - latest - Linux - x86_64 . sh` bash Miniconda3 - latest - Linux - x86_64 . sh `source ~ / . bashrc` #conda 환경 생성 `conda create - y - n lerobot python = 3.10` conda activate lerobot #ffmpeg 설치 `conda install ffmpeg - c conda - forge` # lerobot 설치 `git clone https : // github . com / huggingface / lerobot . git` cd lerobot git checkout v0 . 4.3 pip install - e ".[feetech]"

2.1 로봇 포트 인식

로봇이 노트북 혹은 데스크탑에 연결된 포트를 찾는 과정으로 진행 후 이를 기록해 추후에 사용한다 명령 입력 후 알고싶은 포트를 연결 해제하면 해당 포트가 어떤 포트인지 나온다 ex) `lerobot - find - port`

+ by-ID 경로

위와 같이 포트 번호로 진행하게 되면 추후 다시 진행할 때 포트 번호가 계속 바뀌는 현상이 발생한다 . 이를 해결하기 위해 연결한 기기의 고유 시리얼을 대신 사용하는 방법이 있다 . # by-id 목록 보기 `ls - l / dev / serial / by - id` # 포트를 뺏다꼴다하며 특정 by-id가 실제 어떤 tty인지 확인 `readlink - f / dev / serial / by - id / usb - 1a86_USB_Single_Serial_XXXXXX - if00` # 확인한 기기 시리얼을 이런식으로 인자에 넣어 사용할 수 있다 robot . left_arm_port = / dev / serial / by - id / usb - 1a86_USB_Single_Serial_5A7A055507 - if00 top_cam : { type : opencv , index_or_path : '/dev/v4l/by-id/usb-Alcorlink_Corp._USB_2.0_Camera-video-index0' , width : 640 , height : 480 , fps : 30 }

2.2 모터 번호 인식

모터 번호 인식은 사용하는 모터보드에 연결된 모터의 번호를 인식시켜주는 과정으로 모터 보드 교체 혹은 모터 교체시마다 진행하는 것이 안전하다 ex) 사용하는 로봇과 포트를 인자에 입력 `lerobot -`

```
setup - motors \ - - robot . type = so101_follower \ - - robot . port = /  
dev / ttyACM0
```

2.3 카메라 인식 확인

연결한 카메라의 인덱스 번호를 확인 후 기록한다 `python -c " import cv2 for i in range(10): cap = cv2.VideoCapture(i) if cap.isOpened(): print(f'Camera {i}: OK') cap.release() "` 연결한 카메라가 정상적으로 동작하는지 확인하고 각각의 카메라 번호가 어느 위치의 카메라인지 확인한다 `python -c " import cv2 for i in [0, 2, 4, 6]: cap = cv2.VideoCapture(i) if cap.isOpened(): ret, frame = cap.read() if ret: cv2.imwrite(f'cam_{i}.jpg', frame) print(f'Camera {i}: saved cam_{i}.jpg') cap.release() "`

+ so-100 bimanual 코드 수정

lerobot 에서 제공하는 so_100_bimanual 을 so101 로봇으로 사용하기 위해서는 간단한 코드 수정이 필요하다.

```
lerobot/src/lerobot/teleoperators/bi_so100_leader/bi_so100leader.py 와  
lerobot/src/lerobot/robots/bi_so100_follower/bi_so100follower.py 내부 import  
문과 class 수정을 통해 lerobot 에서 제공되는 bi_so100 을 so101 버전으로 수정할 수 있다  
bi_so100follower.py # 변경 전 from lerobot.robots.so100_follower import  
S0100Follower from lerobot.robots.so100_follower.config_so100_follower import  
S0100FollowerConfig # 변경 후 from lerobot.robots.so101_follower import  
S0101Follower from lerobot.robots.so101_follower.config_so101_follower import  
S0101FollowerConfig # 변경 전 left_arm_config = S0100FollowerConfig ( .. . )  
right_arm_config = S0100FollowerConfig ( .. . ) self.left_arm = S0100Follower  
( left_arm_config ) self.right_arm = S0100Follower ( right_arm_config ) # 변경  
후 left_arm_config = S0101FollowerConfig ( .. . ) right_arm_config =  
S0101FollowerConfig ( .. . ) self.left_arm = S0101Follower ( left_arm_config )  
self.right_arm = S0101Follower ( right_arm_config ) bi_so100leader.py # 변경 전  
from lerobot.teleoperators.so100_leader.config_so100_leader import  
S0100LeaderConfig from lerobot.teleoperators.so100_leader.so100_leader import  
S0100Leader # 변경 후 from  
lerobot.teleoperators.so101_leader.config_so101_leader import  
S0101LeaderConfig from lerobot.teleoperators.so101_leader.so101_leader import  
S0101Leader # 변경 전 left_arm_config = S0100LeaderConfig ( .. . )  
right_arm_config = S0100LeaderConfig ( .. . ) self.left_arm = S0100Leader  
( left_arm_config ) self.right_arm = S0100Leader ( right_arm_config ) # 변경 후  
left_arm_config = S0101LeaderConfig ( .. . ) right_arm_config =  
S0101LeaderConfig ( .. . ) self.left_arm = S0101Leader ( left_arm_config )  
self.right_arm = S0101Leader ( right_arm_config )
```

2.4 로봇 캘리브레이션

로봇의 물리적 값과 소프트웨어 상의 값을 일치시키는 과정으로 사용할 리더암과 팔로워암의 초기 자세를 동일하게 설정하고 진행해야 한다 인자는 사용하는 로봇, 포트 번호, 원하는 캘리브레이션 정보 파일명으로 진행된다 캘리브레이션된 정보는 .cache \ huggingface \ lerobot 안의 robots 와 teleoperators 폴더 안에 지정한 로봇 타입 폴더 속에 로봇 아이디의 json 파일로 저장된다 ex) 팔로워암 `lerobot-calibrate \ --robot.type = so101_follower \ --robot.port =`

```
/dev/ttyACM0 \ --robot.id = my_follower_arm 리더암 lerobot-calibrate \ --
teleop.type = so101_leader \ --teleop.port = /dev/ttyACM1 \ --teleop.id =
my_leader_arm 양팔 lerobot-calibrate \ --robot.type = bi_so100_follower \ --
robot.left_arm_port = /dev/serial/by-id/usb-1a86_USB_Single_Serial_5A7A055507-
if00 \ --robot.right_arm_port = /dev/serial/by-id/usb-
1a86_USB_Single_Serial_5AE6086110-if00 \ --robot.id = bimanual \ --teleop.type
= bi_so100_leader \ --teleop.left_arm_port = /dev/serial/by-id/usb-
1a86_USB_Single_Serial_5A7A058770-if00 \ --teleop.right_arm_port =
/dev/serial/by-id/usb-1a86_USB_Single_Serial_5AE6086008-if00 \ --teleop.id =
bimanual
```

2.5.0 로봇 텔레옴

캘리브레이션이 정상적으로 진행되었는지 확인하는 단계로 인자는 리더암/팔로워암의 로봇 타입, 포트 번호, 캘리브레이션 파일명으로 진행된다 ex) lerobot-teleoperate `--robot.type = so101\_follower` `--robot.port = COM7` `--robot.id = follower\_arm1` `--teleop.type = so101\_leader` `--teleop.port = COM6` `--teleop.id = leader\_arm1`

2.5.1 so-101 bimanual 텔레옴

robot type과 teleop type을 bi_so100_follower/leader로 변경하여 사전에 확인한 카메라 번호와 bimanual calibration 파일과 함께 인자로 입력하여 진행한다 display_data 인자를 통해 화면에 카메라와 로봇의 모터별 움직임을 비교할 수 있지만 카메라가 많고 로봇을 bimanual로 진행하게 되면 지연이 생길 수 있다 lerobot-teleoperate \ --robot.type = bi_so100_follower \ --robot.left_arm_port = /dev/serial/by-id/usb-1a86_USB_Single_Serial_5A7A055507-if00 \ --robot.right_arm_port = /dev/serial/by-id/usb-1a86_USB_Single_Serial_5AE6086110-if00 \ --robot.id = bimanual \ --robot.cameras = "{ \ top_cam: {type: opencv, index_or_path: '/dev/v4l/by-id/usb-Alcorlink_Corp._USB_2.0_Camera-video-index0', width: 640, height: 480, fps: 30}, \ wrist_cam_left: {type: opencv, index_or_path: '/dev/v4l/by-id/usb-170428-_Integrated_Webcam_HD-video-index0', width: 1280, height: 720, fps: 30}, \ wrist_cam_right: {type: opencv, index_or_path: '/dev/v4l/by-id/usb-Sonix_Technology_Co._Ltd._USB_2.0_Camera-video-index0', width: 640, height: 480, fps: 30} \ }" \ --teleop.type = bi_so100_leader \ --teleop.left_arm_port = /dev/serial/by-id/usb-1a86_USB_Single_Serial_5A7A058770-if00 \ --teleop.right_arm_port = /dev/serial/by-id/usb-1a86_USB_Single_Serial_5AE6086008-if00 \ --teleop.id = bimanual \ --display_data = True

2.6 서버 점검 (학습 서버)

```
nvidia-smi python -V python -c "import torch;
print(torch.cuda.is_available())"
```

3. 데이터 수집

3.0 허깅페이스 로그인

데이터 수집 단계에서 수집한 데이터를 허깅페이스에 업로드하고 싶다면 사전에 허깅페이스 로그인을 진행한다 `huggingface - cli login - - token { YOUR_HF_TOKEN } - - add - to - git - credential`

3.1.0 수집 실행

lerobot-record를 통해 데이터 수집을 진행할 수 있고 이때 `dataset.push_to_hub`로 허깅페이스에 수집한 데이터를 업로드할지 선택할 수 있다 데이터셋의 제목은 `dataset.single_task`로 지정한다 또한 중간에 데이터셋 기록을 종료하고 다시 이어서 하고 싶다면 `--resume = true` 인자를 마지막에 붙혀 수집을 이어나갈 수 있다 이때 주의해야 할 점은 카메라들끼리의 해상도와 프레임을 반드시 일치시켜야 한다 `lerobot-record \ --robot.type = bi_so100_follower \ --robot.left_arm_port = /dev/serial/by-id/usb-1a86_USB_Single_Serial_5A7A055507-if00 \ --robot.right_arm_port = /dev/serial/by-id/usb-1a86_USB_Single_Serial_5AE6086110-if00 \ --robot.id = bimanual \ --robot.cameras = "{ \ top_cam: {type: opencv, index_or_path: '/dev/v4l/by-path/pci-0000:00:14.0-usb-0:2.4.4:1.0-video-index0', width: 640, height: 480, fps: 25}, \ wrist_cam_left: {type: opencv, index_or_path: '/dev/v4l/by-path/pci-0000:00:14.0-usb-0:1:1.0-video-index0', width: 640, height: 480, fps: 25}, \ wrist_cam_right: {type: opencv, index_or_path: '/dev/v4l/by-path/pci-0000:00:14.0-usb-0:9.2:1.0-video-index0', width: 640, height: 480, fps: 25} \ }" \ --dataset.repo_id = yubin11/lingbot_bimanual_eval \ --dataset.num_episodes = 10 \ --dataset.single_task = "Pick all the towels from the basket and fold them in half twice" \ --dataset.fps = 25 \ --policy.path = /home/ryu/outputs/train/act_lingbot_bimanual_pick_20260207_234921 \ --dataset.push_to_hub = false`

3.1.1 특정 에피소드 삭제

데이터를 수집하다 보면 데이터에 노이즈가 너무 많거나 오류로 인해 저장에 잘못되는 경우가 있는데 이때 특정 에피소드만 지울 수 있다 `python -m lerobot.scripts.lerobot_edit_dataset \ --repo_id yubin11/lingbot_bimanual_pick \ --root /home/ryu/.cache/huggingface/lerobot \ --operation.type delete_episodes \ --operation.episode_indices "[12]"`

4. 수집 데이터 서버 전송 및 학습

4.1 데이터 전송

빠른 학습을 위해 서버를 사용한다면 `scp` 명령을 통해 서버로 데이터를 전송할 수 있다 `scp -r < YOUR_LOCAL_DATASET_ROOT > \ < YOUR_SERVER_USER > @ < YOUR_SERVER_IP > : < YOUR_SERVER_DATA_ROOT >`

4.2 ACT 학습

ACT는 Lerobot 플랫폼 내에서 진행되는 Imitation Learning으로 Action Chunking with Transformers이다. 따라서 추가적인 설치 없이 Lerobot을 설치했다면 바로 학습이 가능하다 서버에서 진행한다면 사용 가능한 GPU 개수를 확인해 `CUDA_VISIBLE_DEVICES`를 설정하고 GPU, CPU 환경을 확인해 `batch_size`와 `num_workers`를 설정한다. 전송받은 데이터셋의 경로와 이름,

출력경로와 job_name을 설정한다. tolerance_s는 타임스텝 허용 오차이고 steps는 최대 스텝, save_freq는 체크포인트 주기, log_freq는 log를 몇 스텝마다 출력할지이다. 만약 잘못 수집된 에피소드가 데이터셋에 섞여있다면 dataset.episode 인자를 통해 사용할 에피소드만 골라서 사용한다.

만약 기존에 학습한 체크포인트에 추가로 수집한 데이터셋으로 학습을 진행하고 싶다면 policy.type 다음줄에 "--policy.pretrained_path=사전 학습된 체크포인트" 를 추가해 진행하면 된다

```
CUDA_VISIBLE_DEVICES = 0,1 torchrun --nproc_per_node = 2 \ $( which lerobot-  
train ) \ --dataset.repo_id = yubinn11/lingbot_bimanual_pick \ --dataset.root  
= /DATA/tmluser2/yubinn11/lingbot_bimanual_pick \ --policy.type = act \ --  
output_dir = outputs/train/act_lingbot_bimanual_pick_10k \ --job_name =  
act_lingbot_bimanual_pick_10k \ --policy.device = cuda \ --wandb.enable =  
false \ --policy.push_to_hub = false \ --tolerance_s = 0.001 \ --batch_size =  
64 \ --num_workers = 16 \ --steps = 10000 \ --save_freq = 2000 \ --log_freq =  
100 \ --dataset.episodes =  
"[13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32]" \ 2 > &1 | tee  
train_log.txt
```

5. ACT 로컬 추론

5.1 로컬로 체크포인트 이동

학습이 완료된 체크포인트 혹은 중간 검증을 위해 마지막 체크포인트를 로컬로 가져온다 scp -P 10022 -r tmluser@203.252.159.15:~/le/outputs/act_lingbot_pick3_ft_v2/checkpoints/020000/pretrained_model /home/ryu/outputs/train/act_lingbot_pick3_ft_v2/

5.2 로컬에서 체크포인트 추론

act 로 학습된 모델은 데이터 수집때 사용했던 lerobot-record를 통해 추론된다 dataset repo id는 어떤 폴더명으로 추론하며 저장된 정보를 저장할지이고 num_episode는 몇 에피소드를 진행할지, episode_time은 한 에피소드를 얼마나 지속할지, reset time은 한 에피소드 진행 후 환경 초기화 시간이다 lerobot-record \ --robot.type = bi_so100_follower \ --robot.left_arm_port = /dev/serial/by-id/usb-1a86_USB_Single_Serial_5A7A055507-if00 \ --robot.right_arm_port = /dev/serial/by-id/usb-1a86_USB_Single_Serial_5AE6086110-if00 \ --robot.id = bimanual \ --robot.cameras = "{ \ top_cam: {type: opencv, index_or_path: '/dev/v4l/by-path/pci-0000:00:14.0-usb-0:2.4.4:1.0-video-index0', width: 640, height: 480, fps: 25}, \ wrist_cam_left: {type: opencv, index_or_path: '/dev/v4l/by-path/pci-0000:00:14.0-usb-0:1:1.0-video-index0', width: 640, height: 480, fps: 25}, \ wrist_cam_right: {type: opencv, index_or_path: '/dev/v4l/by-path/pci-0000:00:14.0-usb-0:9.2:1.0-video-index0', width: 640, height: 480, fps: 25} \ }" \ --dataset.repo_id = yubinn11/eval_lingbot_pick3_scratch_v2 \ --dataset.num_episodes = 10 \ --dataset.single_task = "Pick all the towels from the basket and fold them in half twice" \ --dataset.fps = 25 \ --dataset.episode_time_s = 300 \ --dataset.reset_time_s = 30 \ --policy.path = /home/ryu/outputs/train/act_lingbot_pick3_scratch_v2 \ --dataset.push_to_hub = false

